

Author

Utkarsh Som

21f1006340

21f1006340@ds.study.iitm.ac.in

I am Utkarsh, and I just graduated from my university and currently working as a Data Scientist, this is my first time working on web application, learnt a lot from the process.

Description

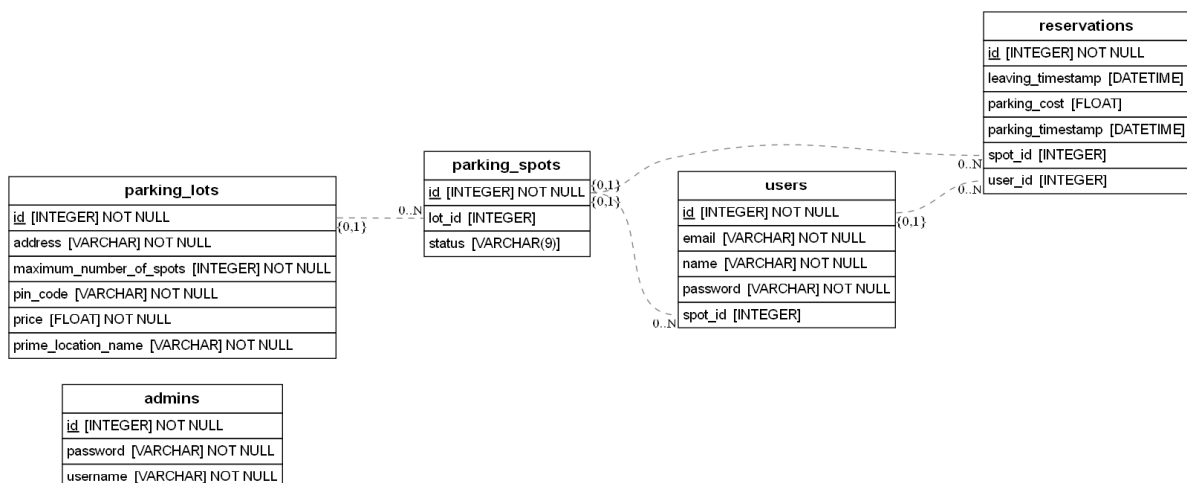
The project requires a fully functional vehicle parking web app, based most on flask backend for backend and jinja for frontend deployment. The app covers all practical applications, like CRUD, cost, history etc, for parking lots from both user and admin end.

I have mainly used AI/LLM for Backend (Flask + DB) - 20% and Styling/UI -8%, used it wherever I was stuck for more than few days and for beautifying my web app.

Technologies used

- **Flask** – Used as web server framework
- **SQLite** – Used for database, to store data as a relational database
- **Flask-SQLAlchemy** – Used to implement my database in the flask application
- **Flask-Login** – Used to handle authentication control
- **Werkzeug** – Used for password hashing for user for added layer of security
- **Flask-WTF** – Used for my webforms
- **email-validator** – Used to check the correctness of emails entered
- **Jinja2** – Used templates for frontend view
- **WTForms** – Used in combination with the Flask_WTF to define html forms in python

DB Schema Design



- Users can have one active spot and multiple reservations
- Lots contain many spots, and each spot can be booked many times
- I have kept the design is simple, secure, and easy to expand
- Also added real-time status tracking and full booking history

Roles are separated (admin and user) with authentication

Architecture and Features

I organized the project by following a clean and modular structure. All the route handlers and application logic are placed inside the ***controllers*** directory, which keeps the backend logic separate from other components. The ***models*** folder contains all the database models using SQLAlchemy, defining the structure and relationships of the data. For the frontend, I used the ***templates*** directory to store all the HTML files, which are rendered using Jinja2.

I built the application using Flask and SQLAlchemy, and included key features like user registration, login, real-time parking spot reservation, and an admin dashboard. To keep the system secure, I implemented password hashing, CSRF protection, and input validation. The frontend uses Bootstrap, which helps make the interface responsive and user-friendly. I focused on writing clean, maintainable code, with a clear separation of logic and proper error handling across the entire app.

Video

https://drive.google.com/file/d/1HFp89FHjAkdNduQi5fzrG6aR_dcfEDve/view?usp=sharing